

BCT 2408 COMPUTER ARCHITECTURE

LAB 3

EARL KINUTHIA SCT 212-0067/2019

E1:

For each code fragment, identify the type of hazard and the registers involved.

a.

LD R1, 0(R2)

DADD R3, R1, R2

- Hazard Type: Read After Write (RAW)
- Registers Involved: R1
- Explanation: The DADD instruction reads R1 before the LD instruction writes to it, leading to a RAW hazard.

b.

MULT R1, R2, R3

DADD R1, R2, R3

- Hazard Type: Write After Write (WAW)
- Registers Involved: R1
- Explanation: Both instructions write to R1. If DADD completes before MULT, it can overwrite the result of MULT, causing a WAW hazard.

c.

MULT R1, R2, R3

MULT R4, R5, R6

- Hazard Type: None
- Explanation: The instructions operate on different registers with no dependencies, so no hazard exists.

d.

DADD R1, R2, R3

SD 2000(R0), R1

- Hazard Type: Read After Write (RAW)

- Registers Involved: R1
- Explanation: The SD instruction reads R1 to store its value, but R1 is being written by the preceding DADD instruction, leading to a RAW hazard.

e.

DADD R1, R2, R3

SD 2000(R1), R4

- Hazard Type: Read After Write (RAW)
- Registers Involved: R1
- Explanation: Here, R1 is used to calculate the memory address in the SD instruction. Since R1 is being written by DADD, there's a RAW hazard concerning the address computation.

E2:

a. Behavior of a 2-Bit Saturating Counter

A 2-bit saturating counter is a finite state machine used in dynamic branch prediction. It has four states:

Assumptions:

- Every other element of x is zero, starting with the first one.
- Initial state of the predictor: Strongly Not Taken (00).

Branch Instruction of Interest:

BNEZ F1, else

Prediction Analysis:

We'll analyze the first few iterations to observe the predictor's behavior.

Current State	Prediction	Branch Outcome	New State
00	Not Taken	Not Taken	00
00	Not Taken	Taken	01
01	Not Taken	Not Taken	00

01	Not Taken	Taken	10
10	Taken	Not Taken	01
10	Taken	Taken	11
11	Taken	Not Taken	10
11	Taken	Taken	11

Observations:

- The predictor mispredicts every time the branch is **taken** because it requires two consecutive taken outcomes to switch its prediction to taken.
- Since the pattern alternates between not taken and taken, the predictor remains in the lower states (00 or 01), leading to frequent mispredictions on taken branches.

b. Consider the following code:

```
for (i=0; i<N;
i++) if (x[i] ==
0)
```

```
y[i] = 0.0;
```

```
else
```

```
y[i] = y[i]/x[i];
```

Assume that the assembly code generated is then:

```
loop: L.D F1, 0(R2)
```

```
L.D F2, 0(R3)
```

```
BNEZ F1, else
```

```
ADD.D F2,
```

```
F0, F0
```

```
BEZ R0, fall
```

```
else: DIV.D F2, F2,
```

```
F1 fall: DADDI R2,
```

```
R2, 8 DADDI R3,
```

```
R3, 8 DSUBI R1,
```

```
R1, 1
```

S.D -8(R3), F2

BNEZ R1,

loop where:

- the value of N is already stored in R1
- the base addresses for x and y are stored in R2 and R3, respectively
- register F0 contains the value 0
- register R0 (always) contains the value 0

Assuming every other element of x has the value 0, starting with the first one, show the outcomes of predictions when a 2-bit saturating counter is used to predict the inner branch BNEZ F1, else. Assume that the initial value of the counter is 00.

Initial Counter Value	Prediction	Outcome if Taken	Outcome if Not Taken
00	Not Taken	Transitions to 01	Stays at 00
01	Not Taken	Transitions to 10	Transitions to 00
10	Taken	Transitions to 11	Transitions to 01
11	Taken	Stays at 11	Transitions to 10